

Improved Cryptanalysis of Forró via Trail Enumeration Puncturing

Soumya Sahoo and Santanu Sarkar

Department of Mathematics, IIT Madras, Chennai, India.*

Abstract

At EUROCRYPT 2025, Flórez-Gutiérrez and Todo introduced a new key-recovery attack on ChaCha based on trail enumeration puncturing, which was later extended to Salsa at ASIACRYPT 2025. In this work, for the first time, we apply the trail enumeration puncturing technique to Forró, a cipher that exhibits stronger confusion and diffusion than both Salsa and ChaCha. Furthermore, we propose the first 3-round differential-linear distinguisher for Forró. As a result, we improve the time complexity of the best-known attack on 5-round Forró by approximately a factor of 2^{11} .

Key Words : Differential-linear distinguisher, Trail enumeration puncturing, Bit Puncturing, Forró.

1 Introduction

Stream ciphers from the Salsa/ChaCha family are widely used in modern operating systems due to their efficiency and strong security. Salsa20 and ChaCha20 have been shown to perform well in lightweight and resource-constrained IoT environments [MAS16, ZSH24]. Several improved ChaCha variants have also been proposed to secure IoT data and sensor communications [MHAM21]. This growing adoption naturally motivates the study of newer ARX-based designs. In this context, the recently proposed stream cipher Forró [CPdCGV⁺23], which shares structural similarities with the ChaCha family, is a promising candidate for future IoT deployments. The designers of Forró claim improved performance over ChaCha on IoT devices. Therefore, a careful security analysis of Forró is essential before its use in emerging operating systems.

Cryptanalysis of Forró. Since Forró is a relatively new design, its public cryptanalytic exploration is still limited. Apart from the differential-linear

*soumyasahoo078@gmail.com, sarkar.santanu.bir1@gmail.com

analysis carried out by the designers [CPdCGV⁺23], only a few independent studies have examined its security so far [SGD24, GWH⁺26]. In their analysis [CPdCGV⁺23], the designers proposed a key-recovery attack of 5-round Forró with a time complexity of 2^{158} , suggesting that, with the existing PNB-based methodology, key recovery beyond 5 rounds remains impractical without falling back to brute force. The follow-up work in [SGD24] revisited this setting using a multi-step approach to improve the 5-round analysis, and later employed the syncopation technique of [WLHL23] to extend the attack to 5.5 rounds. In [GWH⁺26], the authors presented a 5.5-round differential-linear distinguisher but did not provide any key-recovery attacks on forró.

Trail Enumeration Puncturing. A recent work presented at EUROCRYPT 2025 [FGT25] and ASIACRYPT 2025 [FGT26] introduced a new key-recovery method for ChaCha-like ciphers that builds on Walsh spectrum puncturing for linear cryptanalysis [FGT24]. The general intuition is related to probabilistic neutral bits, but the technique differs in two important aspects. First, the key bits treated as neutral, referred to as punctured bits, are not fixed to specific values. Instead, the target bit is approximated by a pseudoboolean function obtained by averaging over all possible assignments of these punctured bits. Second, the approximating function and its correlation are not obtained through experiments. Rather, they are computed analytically using trail enumeration puncturing, where the function is expressed as a sum of dominant linear trails. This approach is applicable when the Walsh spectrum is mainly influenced by a small number of strong linear trails. In practice, this condition is typically satisfied for ARX-based constructions.

Our Contributions. Until now, probabilistic neutral bits (PNBs) have been the most effective tool for the cryptanalysis of Salsa, ChaCha, and Forró. More recently, trail enumeration puncturing significantly improved the attacks on Salsa and ChaCha. However, no attack on Forró based on trail enumeration puncturing had been reported so far. In this work, we present the first application of trail enumeration puncturing to Forró. In addition, we propose the first 3-round differential-linear distinguisher for Forró. By combining this new distinguisher with the partitioning technique and the use of a distillation table, we improve the time complexity of the best-known attack on 5-round Forró by approximately a factor of 2^{11} . A comparison between previous attacks and our results on Forró is provided in Table 1.

2 Preliminaries

2.1 Design of Our Target Cipher

Similar to Salsa and ChaCha, the cipher Forró uses a 4×4 state matrix V^0 consisting of 16 words, each of 32 bits. The state includes eight

Table 1: Comparison of attacks against reduced-round Forró.

Target	Data	Time	Notes	Ref.
Forró 5/256	2^{57}	2^{158}	PNBs w/ Distinguisher	[CPdCGV ⁺ 23]
	$2^{52.36}$	$2^{144.14}$	PNBs w/ Multi-Step	[SGD24]
	$2^{101.6}$	$2^{136.94}$	Puncturing w/ Improved Distinguisher	This Work
	$2^{101.6}$	$2^{133.18}$	Puncturing, Improved Distinguisher w/ Distillation	This Work
Forró 5.5/256	2^{47}	$2^{225.04}$	PNBs w/ Syncopation	[SGD24]

key words $(k_0, k_1, k_2, k_3, k_8, k_9, k_{10}, k_{11})$, four constants (c_0, c_1, c_2, c_3) , two counter words (t_0, t_1) , and two nonce words (t_2, t_3) . The initial and r -th round states are denoted by

$$V^0 = \begin{pmatrix} k_0 & k_1 & k_2 & k_3 \\ t_0 & t_1 & c_0 & c_1 \\ k_8 & k_9 & k_{10} & k_{11} \\ t_2 & t_3 & c_2 & c_3 \end{pmatrix}, \quad V^r = \begin{pmatrix} v_0^r & v_1^r & v_2^r & v_3^r \\ v_4^r & v_5^r & v_6^r & v_7^r \\ v_8^r & v_9^r & v_{10}^r & v_{11}^r \\ v_{12}^r & v_{13}^r & v_{14}^r & v_{15}^r \end{pmatrix}.$$

Each round of Forró consists of four identical **sub-round** operations (see Figure 1). In a sub-round, a five-word vector (p, q, r, s, t) is updated using modular addition, XOR, and rotations as follows:

$$\begin{aligned} s' &= s \boxplus t, & r' &= r \oplus s', & q' &= (q \boxplus r') \lll 10, \\ p' &= p \boxplus q', & t'' &= t \oplus p', & s'' &= (s' \boxplus t'') \lll 27, \\ r'' &= r' \boxplus s'', & q'' &= q' \oplus r'', & p'' &= (p' \boxplus q'') \lll 8. \end{aligned}$$

A major distinction between Forró and the Salsa/ChaCha ciphers is the number of input words used. Forró utilizes 5 input words, whereas Salsa and ChaCha use only 4. The **sub-round** function (Figure 1) is asymmetric since the word t is updated only once, while all other words are updated twice. Similar to the Salsa cipher family, Forró follows a structured approach where columns and diagonals are modified in alternating rounds. Each round function of Forró consists of four **sub-round** operations. During odd rounds, the columns $(v_0, v_4, v_8, v_{12}, v_{16})$, $(v_1, v_5, v_9, v_{13}, v_{17})$, $(v_2, v_6, v_{10}, v_{14}, v_{18})$, and $(v_3, v_7, v_{11}, v_{15}, v_{19})$ are updated. In even rounds, the diagonals $(v_0, v_5, v_{10}, v_{15}, v_{20})$, $(v_1, v_6, v_{11}, v_{16}, v_{21})$, $(v_2, v_7, v_{12}, v_{17}, v_{22})$, and $(v_3, v_8, v_{13}, v_{18}, v_{23})$ undergo updates. After completing all R rounds, the initial state V^0 is added back, resulting in the keystream state Z , which is expressed by $Z = V^0 \boxplus V^R$. Define the state matrix $\bar{Z}$, where $\bar{z}_i = v_i^r$ if $i \in \{4, 5, 6, 7, 12, 13, 14, 15\}$ and $\bar{z}_i = z_i$ otherwise. For the rest of the paper, we use $v_i^r[j]$ to denote the j -th bit of the i -th word of the state V^r .

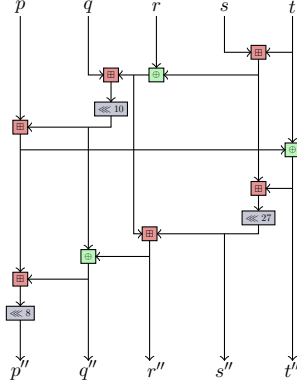


Figure 1: The sub-round function of Forró

2.2 Traditional Attack Idea

Let E be a reduced-round version of an ARX cipher such that $E = E_2 \circ E_1$. Assume that E_1 admits a differential-linear distinguisher from Δ to Γ with correlation ε_d . Let $\bar{Z}$ and $\bar{Z}'$ denote the keystream blocks obtained after removing the modular additions with known input branches. The corresponding key recovery map is then given by

$$f(\bar{Z}, \bar{Z}', K) = \langle \Gamma, E_2^{-1}(\bar{Z}, K) \oplus E_2^{-1}(\bar{Z}', K) \rangle$$

and it is expected to exhibit correlation ε_d for the correct value of K .

Using PNBs, we approximate the key recovery map by fixing key bits that have influence on the correlation. Significant and non-significant key bits are identified through experiments. First, we evaluate $f(\bar{Z}, \bar{Z}', K)$ for a randomly chosen known key. Next, for each index i , we evaluate $f(\bar{Z}, \bar{Z}', K \oplus e_i)$, where e_i denotes the vector that flips the i -th bit of K . If both functions show a correlation above a predefined threshold, the i -th bit is considered non-significant.

We denote the remaining significant key bits by $\bar{K}$ and replace f with a function $g(\bar{Z}, \bar{Z}', \bar{K})$. The function g is identical to f , except that the non-significant bits of K are fixed to constant values. We experimentally measure the correlation between $f(\bar{Z}, \bar{Z}', K)$ and $g(\bar{Z}, \bar{Z}', \bar{K})$, and denote it by ε_a . Assuming independence, the overall correlation for the correct guess of $\bar{K}$ is given by $\varepsilon_d \times \varepsilon_a$.

3 Trail Enumeration Puncturing Attack on Forró

3.1 Trail Enumeration of a Key-Recovery Map

Figure 2 illustrates an example of trail enumeration in a subround of Forró. In Figure 2a, the exact key recovery map is illustrated. In Figure 2b, a corre-

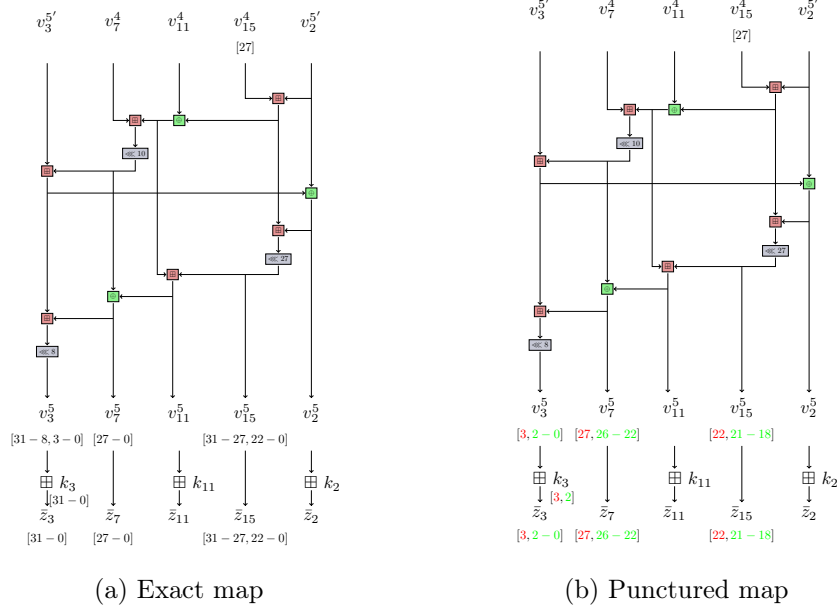


Figure 2: Creation of a punctured map.

sponding punctured map is shown, where bits shown in red are always active, bits shown in green may be active depending on the trail, and all remaining bits are inactive. The objective is to compute $v_{15}^4[27]$ from $(\bar{z}_3, \bar{z}_7, \bar{z}_{11}, \bar{z}_{15}, \bar{z}_2)$, given partial guesses of the subkeys k_3 , k_{11} and k_2 .

The exact key-recovery map for $v_{15}^4[27]$ is

$$(-1)^{v_{15}^4[27]} = f_1(\bar{z}_3[31-0], k_3[31-0], \bar{z}_7[27-0], \bar{z}_{15}[31-27, 22-0]).$$

We begin from the target bit, for example $v_{15}^4[27]$, and enumerate all linear trails that extend from it. The extension is carried out iteratively, processing one modular addition at a time. At each step, we expand every current approximation and discard trails with negligible correlation. After completing a step, approximations that share the same output mask are merged by summing their correlations. This procedure continues until the final round is reached, yielding a partial Walsh spectrum of the key-recovery map.

As illustrated in Figure 2b, we limit the masks of v_3^5 to bits $[3-0]$, of v_7^5 to bits $[27-22]$, and of v_{15}^5 to bits $[22-18]$. Equivalently, all trails activating any bit in $v_3^5[31-8]$, $v_7^5[21-0]$, or $v_{15}^5[31-27, 17-0]$ are discarded. With these constraints, the punctured key-recovery map is given by:

$$(-1)^{v_{15}^4[27]} = (-1)^{\bar{z}_3[3] \oplus k_3[3] \oplus \bar{z}_7[27] \oplus \bar{z}_{15}[22]} \times g_1(\bar{z}_3[2-0], k_3[2], \bar{z}_7[26-22], \bar{z}_{15}[21-18])$$

Note that we do not guess $k_3[3]$ as it only changes the sign of a correlation. In the end, we only guess $k_3[2]$. As a result, we obtain 2^{13} linear trails. The

Table 2: List of $\mathcal{DL}$ distinguisher for Forró

Input Diff	Output Mask	#Rounds	Bias	Ref.
$\Delta v_5^0[18]$	$\Gamma_{15}^2[7]$	2	$2^{-8.04}$	[CPdCGV+23]
$\Delta v_5^0[18]$	$\Gamma_{10}^2[7]$	2	$2^{-8.82}$	[CPdCGV+23]
$\Delta v_5^0[11]$	$\Gamma_{15}^2[0]$	2	$2^{-9.49}$	[CPdCGV+23]
$\Delta v_5^0[11]$	$\Gamma_{10}^2[0]$	2	$2^{-10.88}$	[CPdCGV+23]
$\Delta v_5^0[21, 26, 31], \Delta v_{13}^0[31]$	$\Gamma_6^3[10] \oplus \Gamma_{10}^3[0, 10] \oplus \Gamma_{14}^3[0]$	3	$2^{-7.92}$	This work

puncturing correlation is estimated as the sum of the squared correlations of all these trails. This yields a puncturing correlation of $2^{-2.45}$.

3.2 A new 3-Round DL Distinguisher

The designers of Forró introduced several 2-round distinguishers in [CPdCGV+23]. However, no 3-round distinguisher was provided. Moreover, these 2-round distinguishers are not suitable for key recovery attacks on 5 rounds. In this work, we propose the first 3-round differential-linear (DL) distinguisher for Forró.

Using a MILP model, we first identify a 1-round differential that holds with probability 2^{-5} :

$$(\Delta v_5^0[21, 26, 31], \Delta v_{13}^0[31]) \xrightarrow{\mathcal{D}} (\Delta v_0^1[4, 31], \Delta v_5^1[4, 31], \Delta v_{13}^1[31]).$$

Next, we extend this trail by 1.5 rounds using a differential-linear transition with a forward bias of $2^{-2.92}$:

$$(\Delta v_0^1[4, 31], \Delta v_5^1[4, 31], \Delta v_{13}^1[31]) \xrightarrow[1.5 \text{ round}]{\mathcal{DL}} \Gamma_6^{2.5}[0],$$

Finally, we extend the distinguisher linearly by an additional 0.5 round with probability one, which results in a 3-round DL distinguisher:

$$\Delta v_5^0[21, 26, 31], \Delta v_{13}^0[31] \xrightarrow[3 \text{ round}]{\mathcal{DL}} \Gamma_6^3[10] \oplus \Gamma_{10}^3[0, 10] \oplus \Gamma_{14}^3[0]$$

A summary of the distinguishers proposed by the designers in [CPdCGV+23], together with our newly introduced distinguisher, is presented in Table 2.

3.3 Linear Decomposition of the Key-Recovery Map

The output mask $\Gamma_6^3[10] \oplus \Gamma_{10}^3[0, 10] \oplus \Gamma_{14}^3[0]$ of the newly proposed distinguisher is only partially extended when the correlation equals 1 (see Figure 3). The output parity of the 3-round differential-linear distinguisher is

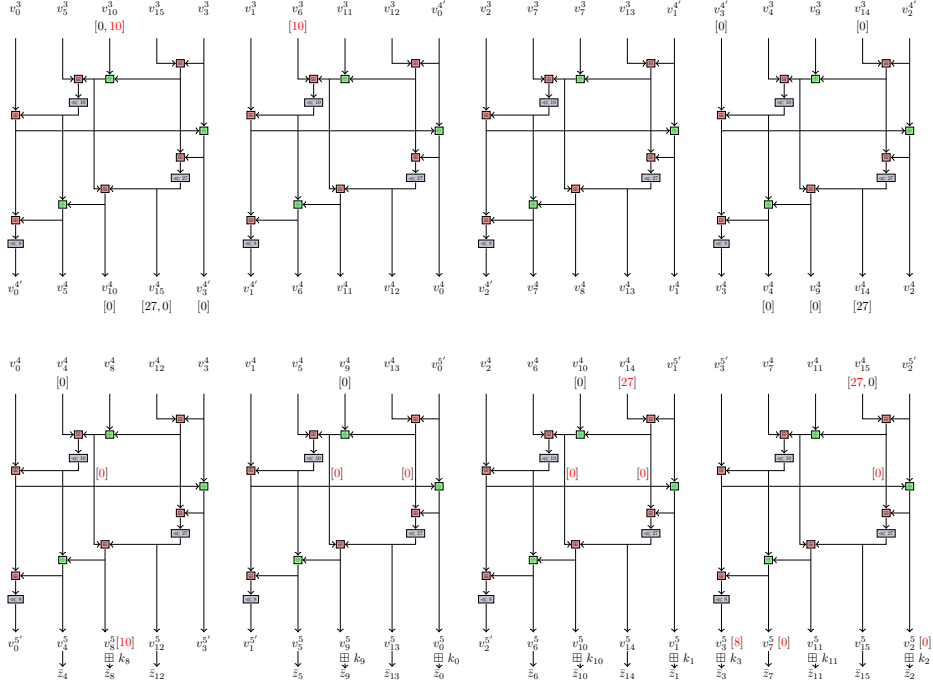


Figure 3: Linear decomposition of the 5-round key-recovery map.

the sum of the bits highlighted in red. We have decomposed the key recovery as a sum of functions $f_i(\bar{Z}, K)$. The functions which involve more than one modular subtraction are:

$$f_1 = (-1)^{v_{15}^4[27]}, \quad f_2 = (-1)^{v_{14}^4[27]}, \quad f_3 = (-1)^{v_6^3[10]}, \quad f_4 = (-1)^{v_{10}^3[10]}$$

The other functions, that can be evaluated by subtracting the secret key from the key stream or its equivalent are:

$$f_5 = (-1)^{v_8^5[10]} \quad \text{and} \quad f_6 = (-1)^{v_3^5[8]}$$

If the least significant bit is active, for example $v_2^5[0]$, the corresponding bit of $\bar{Z}$ must be included in the analysis, whereas the related key bit does not need to be guessed. Therefore, we can safely ignore the functions related to the least significant bits. For the Boolean functions f_5 and f_6 , we adopt a partitioning approach [Leu16, BLT20], as it results in a reduced computational complexity for the key-recovery procedure. In contrast, trail enumeration puncturing [FGT25] is performed for the functions f_1, f_2, f_3 and f_4 .

3.4 Punctured Maps for Key Recovery

For the function f_1 , the punctured map has already been constructed in subsection 3.1, with a puncturing correlation of $2^{-2.45}$. For the function f_2 ,

Table 3: Punctured functions for the 5-round attack

Function	Active Bits	#Trails	ρ^2
f_1	$\bar{z}_7[27, 26, 25, 24, 23, 22], \bar{z}_{15}[22, 21, 20, 19, 18],$ $v_3^5[3, 2, 1, 0] \leftarrow \{\bar{z}_3[3, 2, 1, 0], k_3[3, 2]\}$	2^{13}	$2^{-2.45}$
f_2	$\bar{z}_6[27, 26, 25], \bar{z}_7[3, 2, 1], \bar{z}_{14}[22, 21],$ $v_2^5[3, 2, 1] \leftarrow \{\bar{z}_2[3, 2, 1], k_2[3, 2, 1]\},$ $v_3^5[11, 10, 9] \leftarrow \{\bar{z}_3[11, 10, 9], k_3[11, 10, 9]\}$	2^{13}	$2^{-3.54}$
f_3	$\bar{z}_6[30, 29, 28, 27, 26], \bar{z}_{14}[20, 19, 18, 17], \bar{z}_{15}[20, 19, 18, 15, 14, 13],$ $\bar{z}_{15}[10, 9, 8, 5, 4, 3], \bar{z}_4[10, 9], \bar{z}_5[18, 17], \bar{z}_6[26, 25], \bar{z}_7[2, 1], \bar{z}_{12}[5, 4],$ $v_{10}^5[30, 29, 28, 20, 19, 18] \leftarrow \{\bar{z}_{10}[30, 29, 28, 20, 19, 18], k_{10}[30, 29, 20, 19]\},$ $v_2^5[20, 19, 18] \leftarrow \{\bar{z}_2[20, 19, 18], k_2[20, 19, 18]\},$ $v_{11}^5[20, 19, 18] \leftarrow \{\bar{z}_{11}[20, 19, 18], k_{11}[20, 19, 18]\},$ $v_2^5[10, 9, 8] \leftarrow \{\bar{z}_2[10, 9, 8], k_2[10, 9, 8]\},$ $v_{11}^5[10, 9, 8] \leftarrow \{\bar{z}_{11}[10, 9, 8], k_{11}[10, 9, 8]\},$ $v_0^5[18, 19] \leftarrow \{\bar{z}_0[18, 19], k_0[18, 19]\},$ $v_1^5[26, 25] \leftarrow \{\bar{z}_1[26, 25], k_1[26, 25]\},$ $v_2^5[2, 1] \leftarrow \{\bar{z}_2[2, 1], k_2[2, 1]\},$ $v_3^5[10, 9] \leftarrow \{\bar{z}_3[10, 9], k_3[10, 9]\}$	4×2^{13}	$2^{-13.51}$
f_4	$\bar{z}_7[10, 9, 8, 7], \bar{z}_{15}[5, 4, 3, 2], \bar{z}_7[5, 4, 3, 2, 1, 0], \bar{z}_{15}[0], \bar{z}_{14}[10, 9, 8, 7, 5, 4, 3],$ $\bar{z}_{13}[10, 9, 8, 7, 5, 4, 3], \bar{z}_4[18, 17, 16], \bar{z}_5[26, 25, 24], \bar{z}_6[2, 1], \bar{z}_7[10, 9]$ $v_3^5[18, 17, 16, 15] \leftarrow \{\bar{z}_3[18, 17, 16, 15], k_3[18, 17, 16, 15]\}$ $v_3^5[13, 12, 11, 10, 9] \leftarrow \{\bar{z}_3[13, 12, 11, 10, 9], k_3[13, 12, 11, 10, 9]\}$ $v_1^5[10, 9, 8] \leftarrow \{\bar{z}_1[10, 9, 8], k_1[10, 9, 8]\}$ $v_{10}^5[10, 9, 8] \leftarrow \{\bar{z}_{10}[10, 9, 8], k_{10}[10, 9, 8]\}$ $v_0^5[10, 9, 8] \leftarrow \{\bar{z}_0[10, 9, 8], k_0[10, 9, 8]\}$ $v_9^5[10, 9, 8] \leftarrow \{\bar{z}_9[10, 9, 8], k_9[10, 9, 8]\}$ $v_0^5[26] \leftarrow \{\bar{z}_0[26], k_0[26]\}$ $v_1^5[2] \leftarrow \{\bar{z}_1[2], k_1[2]\}$ $v_2^5[10, 9] \leftarrow \{\bar{z}_2[10, 9], k_2[10, 9]\}$ $v_3^5[26, 25] \leftarrow \{\bar{z}_3[26, 25], k_3[26, 25]\}$ $v_{11}^5[18, 17] \leftarrow \{\bar{z}_{11}[18, 17], k_{11}[18, 17]\}$	5×2^{13}	$2^{-18.97}$

the punctured key-recovery map is given by:

$$f_2 = (-1)^{v_{14}^4[27]} \approx (-1)^{\bar{z}_2[3] \oplus k_2[3] \oplus \bar{z}_3[11] \oplus k_3[11] \oplus \bar{z}_6[27] \oplus \bar{z}_7[3] \oplus \bar{z}_{14}[22]} \times \\ g_2(\bar{z}_2[2, 1], k_2[2, 1], \bar{z}_3[10, 9], k_3[10, 9], \bar{z}_6[26, 25], \bar{z}_7[2, 1], \bar{z}_{14}[21])$$

with a correlation $2^{-3.54}$. For the functions f_3 and f_4 , we first derive the following linear approximations:

$$f_3 = (-1)^{v_6^3[10]} = f_3'(v_6^4[20-16], v_{11}^4[10-17, 10-7], v_{12}^4[10-7])$$

$$f_4 = (-1)^{v_{10}^3[10]} = f_4'(v_3^4[18-16], v_9^4[10-8], v_{10}^4[10-7], v_{15}^4[10-7, 5-2])$$

These approximations have correlations of $2^{-0.21}$ and $2^{-2.1}$, respectively. We then extend each linear mask from four rounds to five rounds for both

functions, which yields linear masks on $\bar{Z}$ and the key K . The resulting punctured key-recovery maps for f_3 and f_4 are listed in Table 3, with puncturing correlations of $2^{-13.51}$ and $2^{-18.97}$, respectively.

3.5 Attack Complexity

The correlation of our newly found differential-linear distinguisher (Table 2) is $2^{-7.92}$. When N' pairs are used, we assume that the observed correlation follows a normal distribution $\mathcal{N}(2^{-7.92}, 1/N')$ for the correct key guess. For an incorrect key guess, the distribution is $\mathcal{N}(0, 1/N')$. Let ρ^2 denote the product of all puncturing correlations from f_1 to f_6 . The total data complexity N is estimated as $N = N' \times \rho^{-4}$. To evaluate the punctured maps from f_1 to f_4 , we guess the following 32 key bits:

$$k_0[8, 9, 19], \quad k_1[8, 9, 25], \quad k_2[1, 2, 8, 9, 18, 19], \quad k_3[2, 9, 10, 11, 12, 15, 16, 17, 25], \\ k_9[8, 9], \quad k_{10}[8, 9, 19, 29], \quad k_{11}[8, 9, 17, 18, 19]$$

The corresponding puncturing correlation is $2^{-3.54} \times 2^{-2.45} \times 2^{-13.51} \times 2^{-18.97} = 2^{-38.47}$.

We apply the partitioning technique for f_5 and f_6 . For each decomposition, a 2-bit subkey is guessed. Therefore, it is sufficient to guess the following 4 key bits: $k_8[9, 8]$ and $k_3[7, 6]$. Using the partitioning technique, the parity can be computed with probability $(3/4)^2 \approx 2^{-0.83}$. In total, 36 key bits are guessed. The overall puncturing correlation is $\rho^2 = 2^{-38.47-0.83} = 2^{-39.3}$.

If $N' = 2^{23}$, the success probability with a 36-bit key advantage is almost one. Thus, the required data complexity is

$$N = N' \times 2^{39.3 \times 2} = 2^{101.6}.$$

In the partitioning technique, the available pairs are immediately determined depending on the key guess. Therefore, the time complexity is estimated as

$$2N \times 2^{-0.83 \times 2} \times 2^{36} = 2^{136.94}.$$

3.6 Improving the attack complexity using Distillation Table

The distillation table is a classical method used to speed up key recovery in linear attacks [Mat94]. In this approach, information obtained from keystream pairs is stored in a table, and key guesses are tested using this table. If the size of the distillation table is smaller than the number of available pairs, the overall time complexity can be reduced. A keystream block of Forró contains 512 bits, which would normally result in a very large distillation table. To avoid this issue, the distillation table technique is used with the existing attack strategy. In the first phase, we evaluate selected functions in the decomposition. This step reduces the number of keystream bits involved in the attack. When the remaining size becomes manageable, we then apply the distillation table method.

First Phase. The two punctured functions f_3 and f_4 depend on many bits of $\bar{Z}$. Therefore, they are not suitable for the distillation table and are evaluated in the first phase. In total, 30 key bits are guessed.

The time complexity of the First-Phase is estimated as

$$T_1 = 2N \times 2^{30} = 2^{132.6}.$$

Second Phase. After the first phase, the remaining functions are f_1 , f_2 , f_5 , and f_6 . When puncturing is applied to f_1 and f_2 , function f_1 depends on 12 bits of $\bar{Z}$ and 12 bits of $\bar{Z}'$, while function f_2 depends on 9 bits of $\bar{Z}$ and 9 bits of $\bar{Z}'$. For functions f_5 and f_6 , partitioning is used with a 2-bit tail. In this case, each function depends on 2 bits of $\bar{Z}$ and 2 bits of $\bar{Z}'$.

Therefore, a distillation table with a dimension of

$$9 \times 2 + 12 \times 2 + 2 \times 2 + 2 \times 2 + 30 = 80$$

is sufficient.

Using the output of First Phase, we construct the distillation table with a cost of $N \times 2^{30} = 2^{131.6}$. We then evaluate the remaining functions one by one. For example, first evaluating f_1 costs $2^{80} \times 2^1 = 2^{81}$ and reduces the table size to $9 \times 2 + 2 \times 2 + 2 \times 2 + 31 = 57$. Next, evaluating f_2 costs $2^{57} \times 2^4 = 2^{61}$ and reduces the table size to $2 \times 2 + 2 \times 2 + 35 = 43$. Similarly, we will repeat this process for the remaining functions f_5 and f_6 , and the corresponding costs are 2^{45} and 2^{43} . Therefore, the time complexity of Second-Phase is estimated as

$$T_2 = 2^{131.6} + 2^{81} + 2^{61} + 2^{45} + 2^{43}$$

The total time complexity of both phases is

$$T_1 + T_2 = 2^{133.18}.$$

4 Future Work

In this work, we improve the key-recovery attack on 5-round Forró by applying the trail enumeration puncturing technique instead of the traditional PNB-based approach. In the same spirit, we believe that this method can be extended to improve the existing attack on 5.5 rounds and potentially lead to an attack on 6 rounds.

References

- [BLT20] Christof Beierle, Gregor Leander, and Yosuke Todo. Improved differential-linear attacks with applications to arx

- ciphers. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, volume 12172 of *Lecture Notes in Computer Science*, pages 329–358, Santa Barbara, CA, USA, 2020. Springer.
- [CPdCGV⁺23] Murilo Coutinho, Iago Passos, Juan del Carmen Grados Vasquez, Santanu Sarkar, Fábio L. L. Mendonça, Rafael Timóteo de Sousa Jr., and Fábio Borges. Latin Dances Reloaded: Improved Cryptanalysis Against Salsa and ChaCha, and the Proposal of Forró. *J. Cryptol.*, 36(3):18, 2023.
- [FGT24] Antonio Flórez-Gutiérrez and Yosuke Todo. Improving linear key recovery attacks using walsh spectrum puncturing. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 187–216, Cham, 2024. Springer Nature Switzerland.
- [FGT25] Antonio Flórez-Gutiérrez and Yosuke Todo. Improved cryptanalysis of chacha: Beating pnbs with bit puncturing. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 427–457, Cham, 2025. Springer Nature Switzerland.
- [FGT26] Antonio Flórez-Gutiérrez and Yosuke Todo. Divide-and-conquer trail enumeration puncturing: Application to salsa and chacha. In *Advances in Cryptology – ASIACRYPT 2025*, Lecture Notes in Computer Science. Springer Nature Singapore, 2026.
- [GWH⁺26] Xinxin Gong, Qingju Wang, Yonglin Hao, Lin Jiao, and Xichao Hu. Persistence of hourglass(-like) structure: Improved differential-linear distinguishers for several arx ciphers. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 317–351, Singapore, 2026. Springer Nature Singapore.
- [Leu16] Gaëtan Leurent. Improved differential-linear cryptanalysis of 7-round chaskey with partitioning. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 344–371, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [MAS16] Bodhisatwa Mazumdar, Sk. Subidh Ali, and Ozgur Sinanoglu. A compact implementation of salsa20 and its power analysis vulnerabilities. *ACM Trans. Des. Autom. Electron. Syst.*, 22(1), November 2016.

- [Mat94] Mitsuru Matsui. The first experimental cryptanalysis of the data encryption standard. In Yvo G. Desmedt, editor, *Advances in Cryptology — CRYPTO '94*, pages 1–11, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
- [MHAM21] M. S. Mahdi, N. F. Hassan, and G. H. Abdul-Majeed. An improved chacha algorithm for securing data on iot devices. *SN Applied Sciences*, 3(429):1–12, 2021.
- [SGD24] Nitin Kumar Sharma, Hirendra Kumar Garai, and Sabyasachi Dey. Breaching Forró’s Security With Differential-Linear Foray. *IEEE Access*, 12:99175–99182, 2024.
- [WLHL23] Shichang Wang, Meicheng Liu, Shiqi Hou, and Dongdai Lin. Moving a Step of ChaCha in Syncopated Rhythm. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 273–304, Cham, 2023. Springer Nature Switzerland.
- [ZSH24] I. Zerraza, Z. A. Seghir, and M. Hemam. An efficient lightweight authentication and access control for iot edge devices. *International Journal of Safety and Security Engineering*, 14(3):807–813, 2024.